SHERLOCK

Security Review For M0

Collaborative Audit Prepared For: **M0**
Lead Security Expert(s): **KupiaSec**
LZ_security
Date Audited: **December 8 - December 16, 2025**

Introduction

M0's Limit Order Protocol is designed to streamline in and out flows from stablecoins built on the M0 platform by providing an intent-based solution that allows arbitrary swaps for same and crosschain routes. It is built on M0's V2 crosschain infrastructure which supports custom messages for the protocol and supports arbitrary messaging solutions.

Scope

Repository: m0-foundation/liquidity-delivery

Audited Commit: 15a972bc51a559824b699461c9cdd706badd8491

Final Commit: 53510e6dbf5ca5ae2fae9fd3f79c7f4f75210566

Files:

- evm/src/interfaces/IOrderBook.sol
- evm/src/OrderBook.sol
- svm/programs/order_book/src/constants.rs
- svm/programs/order_book/src/error.rs
- svm/programs/order_book/src/instructions/admin.rs
- svm/programs/order_book/src/instructions/claim_refund.rs
- svm/programs/order_book/src/instructions/configure_destination.rs
- svm/programs/order_book/src/instructions/fill.rs
- svm/programs/order_book/src/instructions/initialize.rs
- svm/programs/order_book/src/instructions/mod.rs
- svm/programs/order_book/src/instructions/open.rs
- svm/programs/order_book/src/instructions/report_fill.rs
- svm/programs/order_book/src/instructions/request_cancel.rs
- svm/programs/order_book/src/lib.rs
- svm/programs/order_book/src/state/mod.rs
- svm/programs/order_book/src/state/orders.rs
- svm/programs/order_book/src/utils.rs

Final Commit Hash

53510e6dbf5ca5ae2fae9fd3f79c7f4f75210566

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
1	2	12

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue H-1: There is no status check in `fillOrder()` of EVM [RESOLVED]

Source: https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/issues/18

Vulnerability Detail

There is no check for order status in `fillOrder()` function of EVM.

Attacker can drain protocol by exploiting this vulnerability.

Attack Scenario:

1. Alice creates the order.
2. Alice requests to cancel the order and she receives input tokens
3. Alice fills order and receives input tokens again and output token, too

```
File: evm/src/OrderBook.sol
399:         // Validate fill data
400:         if (chainId != orderData_.destChainId) revert
 ↪  InvalidDestinationChain();
401:         if (uint256(orderData_.fillDeadline) < block.timestamp) revert
 ↪  OrderExpired();
402:         if (orderData_.version != VERSION) revert InvalidOrderVersion();
403:         if (fillerParams_.amountOutToFill == 0) revert FillAmountZero();
```

```
File: svm/programs/order_book/src/instructions/fill.rs
138:         // Validate the order is in a fillable state
139:         require!(
140:             self.order.data.status == OrderStatus::Created,
141:             OrderBookError::OrderNotFillable
142:         );
```

Code Snippet

https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/blob/2b2f63c97c402238343c90b5dcfdd15da8bc11e7/liquidity-delivery/evm/src/OrderBook.sol#L388

Tool Used

Manual Review

Recommendation

Add check for status in EVM.

```
        if (chainId != orderData_.destChainId) revert InvalidDestinationChain();
+       if (chainId == orderData_.originChainId && $.localOrders[orderId_].status
 ↪  != OrderStatus.Created && orderData_.status != OrderStatus.CancelRequested)
 ↪  revert OrderCanceled();
        if (uint256(orderData_.fillDeadline) < block.timestamp) revert
         ↪  OrderExpired();
        if (orderData_.version != VERSION) revert InvalidOrderVersion();
        if (fillerParams_.amountOutToFill == 0) revert FillAmountZero();
```

```
        // Validate the order is in a fillable state
        require!(
-           self.order.data.status == OrderStatus::Created,
+           self.order.data.status == OrderStatus::Created ||
 ↪  self.order.data.status == OrderStatus::CancelRequested,
            OrderBookError::OrderNotFillable
        );
```

Issue M-1: Vulnerability in `initialize()` function [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/issues/20

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The `initialize()` function creates a `global_account` using a deterministic PDA derived from `GLOBAL_SEED`, with no access control. This allows any user to front-run the protocol team's initialization and become the admin.

Vulnerability Detail

The `global_account` PDA is derived using only `GLOBAL_SEED` as a seed. Since the `initialize()` function has no access control restrictions, any user can call it. If an attacker calls `initialize()` before the protocol team, they will become the admin of the `global_account`. Once initialized, the account can't be re-initialized, and the protocol team loses control permanently.

```
File: svm/programs/order_book/src/instructions/initialize.rs
12:     #[account(
13:         init,
14:         payer = admin,
15:         space = ANCHOR_DISCRIMINATOR_SIZE + OrderBookGlobal::INIT_SPACE,
16:         seeds = [GLOBAL_SEED],
17:         bump
18:     )]
19:     pub global_account: Account<'info, OrderBookGlobal>,
```

Impact

Protocol loses control and should deploy a new program.

Code Snippet

https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/blob/2b2f63c97c402238343c90b5dcfdd15da8bc11e7/liquidity-delivery/svm/programs/order_book/src/instructions/initialize.rs#L16

Tool Used

Manual Review

Recommendation

Add access control to `initialize()` function by set an `ADMIN_PUBKEY` in the code.

Discussion

Oighty

We've used this format before and haven't had problems because we typically deploy + initialize together. This was done to simplify testing and allow for multiple deployments of a program with different owners. That being said, I think your suggestion makes sense in this case.

ith-harvey

If we always plan on deploying & initializing simultaneously, adding an admin variable in the contract itself will only cause testing and deployment headaches.

Suggest we ignore.

Issue M-2: `order_token_in_ata` is not closed after order completion, permanently locking rent SOL [RESOLVED]

Source: https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/issues/32

Summary

`OpenOrder` creates/reuses an `order_token_in_ata` per `order` (rent-exempt SOL paid by the `payer`). However, in the order completion paths (`claim_refund` / `fill` / `report_fill`) the program only transfers out SPL tokens and never closes the ATA to return its SOL balance, causing the `payer`'s rent SOL to become permanently unrecoverable.

Vulnerability Detail

- `OpenOrder` uses `init_if_needed` to create `order_token_in_ata`, with `associated_token::authority = order` and `payer = payer` (`svm/programs/order_book/src/instructions/open.rs:94`).
- The authority of `order_token_in_ata` is the `order` PDA. External users cannot sign to execute the SPL Token `CloseAccount` instruction, so if the program does not provide a close path, the rent-exempt lamports remain stuck in the ATA forever.
- The ATA is not closed in any order-finalization flow:
  - `claim_refund`: transfers `order_token_in_ata.amount` to `sender_token_in_ata` and returns (`svm/programs/order_book/src/instructions/claim_refund.rs:124`).
  - `fill` / `report_fill`: when the order is fully filled, status is set to `Completed` and tokens are moved, but `order_token_in_ata` is still not closed (`svm/programs/order_book/src/instructions/fill.rs:147`, `svm/programs/order_book/src/instructions/report_fill.rs:110`).

Impact

- Direct financial loss: each order locks at least one SPL Token account's rent-exempt SOL (paid by `payer`) that cannot be reclaimed by `payer/sender`.

Code Snippet

https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/tree/main/liquidity-delivery/svm/programs/order_book/src/instructions/open.rs#L94

https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/tree/main/liquidity-delivery/svm/programs/order_book/src/instructions/claim_refund.rs#L124

Tool Used

Manual Review

Recommendation

- In all paths where `order_token_in_ata` is no longer needed (at minimum: `claim_refund`, full-fill `fill`, full-fill `report_fill`), after token transfers and once the token balance is 0, CPI into SPL Token `CloseAccount` to close `order_token_in_ata` and return its lamports to a clearly defined recipient (recommended: `order.data.sender` or the original `payer`).

Issue L-1: There is no escrow mechanism for solver [RESOLVED]

Source: https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/issues/19

Summary

In the `fillOrder` function, funds are transferred to the recipient before the match is confirmed on the `srcChain`.

Vulnerability Detail

If the `reportFill` function fails on the `srcChain`, the solver will not receive the funds. However, the funds on the `dstChain` cannot be returned because they have already been transferred to the recipient. The current implementation attempts to mitigate this risk by setting a `finalityBuffer`. However, if this value is not sufficiently large, or in the case of large transactions, it is still possible for the attacker to steal the solver's funds by filling the block during the `finalityBuffer` period on the `srcChain`.

```
388:     function _fillOrder(
             ...
449:         IERC20(orderData_.tokenOut.toAddress()).safeTransferExactFrom(
450:             msg.sender,
451:             orderData_.recipient.toAddress(),
452:             uint256(amountOutToFill_)
453:         );
```

Attack path:

1. Alice (the attacker) creates an order on the srcChain.
2. Bob (the solver) fills the order on the dstChain while Alice cancels her order simultaneously.
3. Alice fills blocks during the finalityBuffer on the srcChain.
4. Alice calls claimRefund.

Impact

The solver could lose funds.

Code Snippet

https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/blob/main/liquidity-delivery/evm/src/OrderBook.sol#L449-L453

Tool Used

Manual Review

Recommendation

Consider escrowing the solver's funds within the protocol and only transferring the funds after the match is confirmed on the `srcChain`.

Discussion

Oighty

I agree there is the possibility of the solver losing funds if fill reports are not received within the `finalityBuffer`. I'm not sure that lack of an ”escrow” is the right description here. We have an alternative design that requires cancels/refunds to be claimed from the destination chain that allows us to remove the `finalityBuffer` and avoid this problem. We will likely implement this.

Issue L-2: Recipient should be different with solver [RESOLVED]

Source: https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/issues/23

Summary

If the recipient of an order is the same as the solver, the order cannot be filled.

Vulnerability Detail

In the `safeTransferExactFrom` function, if the `from` and `to` addresses are the same, the balance delta after the transfer is zero rather than the specified `value`. As a result, this order cannot be filled.

```
OrderBook.sol
449:         IERC20(orderData_.tokenOut.toAddress()).safeTransferExactFrom(
450:             msg.sender,
451:             orderData_.recipient.toAddress(),
452:             uint256(amountOutToFill_)
453:         );
TransferHelper.sol
29:     function safeTransferExactFrom(IERC20 token, address from, address to,
↪  uint256 value) internal {
30:         uint256 balanceBefore = token.balanceOf(to);
31:         safeTransferFrom(token, from, to, value);
32:         require(token.balanceOf(to) - balanceBefore >= value, "STFE");
33:     }
```

Impact

1. If the user enters the same recipient and solver, the `fillOrder` function will be DoSed.
2. If the user attempts to fill their own order to bypass the cancel delay, the DoS could occur because the recipient and solver may be the same.

Code Snippet

https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/blob/main/liquidity-delivery/evm/src/OrderBook.sol#L449 https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/blob/main/liquidity-delivery/evm/src/OrderBook.sol#L179

Tool Used

Manual Review

Recommendation

Discussion

Oighty

Agree that this is an edge case. I don't think there is much impact here since the order can be refunded after the fill deadline passes and the user is only able to lock their own funds up. However, it wouldn't hurt to add a check to prevent this.

Issue L-3: User can't claim refund immediately when `VERSION` is upgraded [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/issues/24

This issue has been acknowledged by the team but won't be fixed at this time.

Vulnerability Detail

When `VERSION` is upgraded, orders are useless because they can't be filled by following check. However, users must still wait for finality buffer time after cancel request.

```
File: src/OrderBook.sol
402:         if (orderData_.version != VERSION) revert InvalidOrderVersion();
```

Impact

User can't claim refund even order is useless.

Code Snippet

https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/blob/2b2f63c97c402238343c90b5dcfdd15da8bc11e7/liquidity-delivery/evm/src/OrderBook.sol#L355

Tool Used

Manual Review

Recommendation

Add check which allows user to claim refund in such case.

Discussion

Oighty

This might be a slight UX improvement, but I think it's ok that they have to wait. We will likely want to pause the contracts when upgrading anyways so the ”immediate refund claim” won't happen in practice.

Issue L-4: Inconsistent check for `finaltiyBuffer` time [RESOLVED]

Source: https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/issues/26

Vulnerability Detail

When `order.fillDeadline + finalityBuffer == block.timestamp`, EVM reverts, but SVM doesn't revert.

```
File: evm/src/OrderBook.sol
339:             if (uint256(order.fillDeadline) + finalityBuffer_ >=
 ↪  block.timestamp) revert FinalityPending();
340:         }
343:             if (uint256(order.cancelRequestedAt) + finalityBuffer_ >=
 ↪  block.timestamp) revert FinalityPending();
344:         } else {
```

```
File: svm/programs/order_book/src/instructions/claim_refund.rs
108:                 current_timestamp >= order.fill_deadline + finality_buffer,
113:                 current_timestamp >= order.cancel_requested_at +
 ↪  finality_buffer,
```

Impact

User can't claim refund at boundary time in EVM version.

Code Snippet

https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/blob/2b2f63c97c402238343c90b5dcfdd15da8bc11e7/liquidity-delivery/evm/src/OrderBook.sol#L339

Tool Used

Manual Review

Recommendation

```
-           if (uint256(order.fillDeadline) + finalityBuffer_ >= block.timestamp)
 ↪  revert FinalityPending();
+           if (uint256(order.fillDeadline) + finalityBuffer_ > block.timestamp)
 ↪  revert FinalityPending();
```

```
        } else if (order.status == OrderStatus.CancelRequested) {
            // If the order is in CancelRequested status,
            // it can only be refunded if the refund was requested at least
             ↪  finality buffer ago
-           if (uint256(order.cancelRequestedAt) + finalityBuffer_ >=
 ↪  block.timestamp) revert FinalityPending();
+           if (uint256(order.cancelRequestedAt) + finalityBuffer_ >
 ↪  block.timestamp) revert FinalityPending();
```

Discussion

Oighty

Agree that the checks should match.

Issue L-5: `GaslessOrderParams.version` is not used [RESOLVED]

Source: https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/issues/27

Vulnerability Detail

There is no check for `orderParams_.version == VERSION`, even `GaslessOrderParams` structure has `version` field. Although user wants to create order for specific version and then contract is upgraded to new version, order is created.

```
struct GaslessOrderParams {
    uint16 version;
    address sender;
    uint64 nonce;
```

```
File: evm/src/OrderBook.sol
252:         // Verify origin chain and sender nonce
253:         if (orderParams_.originChainId != chainId) revert InvalidOriginChain();
254:         OrderBookStorageStruct storage $ = _getOrderBookStorageLocation();
255:         // Requiring a nonce in the order provides replay protection for the
 ↪  sender
256:         if (orderParams_.nonce != $.senderNonces[orderParams_.sender]) revert
 ↪  InvalidNonce();
```

Code Snippet

https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/blob/2b2f63c97c402238343c90b5dcfdd15da8bc11e7/liquidity-delivery/evm/src/interfaces/IOrderBook.sol#L147

Tool Used

Manual Review

Recommendation

Add check for version or remove version field.

Issue L-6: There is no `recipient` check in SVM [RESOLVED]

Source: https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/issues/28

Vulnerability Detail

In `_openOrder()` of EVM, there is recipient check.

```
File: evm/src/OrderBook.sol
179:         if (orderParams_.recipient == bytes32(0)) revert InvalidRecipient();
```

However, there is no such check in SVM.

```
File: svm/programs/order_book/src/instructions/open.rs
111:     fn validate(&self, params: &OrderParams) -> Result<()> {
```

Code Snippet

https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/blob/2b2f63c97c402238343c90b5dcfdd15da8bc11e7/liquidity-delivery/svm/programs/order_book/src/instructions/open.rs#L111

Tool Used

Manual Review

Recommendation

Add check for recipient.

Issue L-7: Inconsistent version could allow order filling [RESOLVED]

Source: https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/issues/29

Vulnerability Detail

If source chain's version is upgraded but destination chain's version is not upgraded, then user can fill the source chain's orders in destination chain.

```
File: evm/src/OrderBook.sol
402:         if (orderData_.version != VERSION) revert InvalidOrderVersion();
```

Impact

User can fill on destination chain's order, although user can't fill on source chain's order.

Code Snippet

https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/blob/2b2f63c97c402238343c90b5dcfdd15da8bc11e7/liquidity-delivery/evm/src/OrderBook.sol#L402

Tool Used

Manual Review

Recommendation

Upgrade all chain's version at once.

Discussion

Oighty

When upgrades are done, the intent is to do them simultaneously. The reason there is a version is to prevent issues with orders created under a previous version if the order layout changes. I'm not sure I understand the specific issue you are claiming here, but maybe an example will help:

- Assume an order is made on chain A when it has `VERSION = 1` with destination of chain B which is also on `VERSION = 1`
- Then, chain A is upgraded `VERSION = 2` now.

- Since the order version is `1` and chain B `VERSION = 1`, it is still possible to fill the order on the destination. This should be fine because it's in the format that chain B expects.
- However, chain B still has to send a fill report back to chain A, which is on a new version. Are you saying that the fill report may fail if the expected layout or logic is changed in the upgrade?

My preferred approach for this would be to add a `pause` function, which can stop new orders, fills, and refunds. That way we can pause the contracts and perform upgrades together without having these kind of mismatches.

Issue L-8: Inconsistent check for `fillDeadline` in cancel request function [RESOLVED]

Source: https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/issues/30

Vulnerability Detail

In EVM, when `fillDeadline = block.timestamp` user can request cancel order, but SVM prevents this.

```
File: evm/src/OrderBook.sol
309:         // Validate that the order can be cancelled and the caller is the
↪  sender
310:         if (order.status != OrderStatus.Created) revert InvalidOrderStatus();
311:         if (uint256(order.fillDeadline) < block.timestamp) revert
↪  OrderExpired();
```

```
File: svm/programs/order_book/src/instructions/request_cancel.rs
31:         // Validate the fill deadline is in the future, otherwise, there is no
↪  need to cancel
32:         if order.fill_deadline <= Clock::get()?.unix_timestamp as u64 {
33:             return err!(OrderBookError::OrderExpired);
34:         }
```

Code Snippet

https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/blob/2b2f63c97c402238343c90b5dcfdd15da8bc11e7/liquidity-delivery/evm/src/OrderBook.sol#L311

Tool Used

Manual Review

Recommendation

Update check of EVM.

```
        // Validate that the order can be cancelled and the caller is the sender
        if (order.status != OrderStatus.Created) revert InvalidOrderStatus();
-       if (uint256(order.fillDeadline) < block.timestamp) revert OrderExpired();
+       if (uint256(order.fillDeadline) <= block.timestamp) revert OrderExpired();
```

Issue L-9: finalityBuffer Setting Delay [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/issues/31

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

When the `finalityBuffer` changes by more than its current value, the change is applied immediately. However, if it decreases by even 1, the application is delayed by the amount of the previous finalityBuffer. It is sufficient to delay by the amount of `finalityBuffer - effectiveFinalityBuffer`.

Vulnerability Detail

```
558:         destination.newFinalityBufferEffectiveTimestamp = finalityBuffer_ <
↪  effectiveFinalityBuffer
559:             ? uint64(block.timestamp) + effectiveFinalityBuffer
560:             : uint64(block.timestamp);
```

Impact

The setting of the finalityBuffer is delayed more than necessary.

Code Snippet

https://github.com/sherlock-audit/2025-11-highway-nov-24th/blob/main/sc-highway/src/DexDepositRouterUpgradeable.sol#L559

Tool Used

Manual Review

Recommendation

```
558:         destination.newFinalityBufferEffectiveTimestamp = finalityBuffer_ <
↪  effectiveFinalityBuffer
-559:             ? uint64(block.timestamp) + effectiveFinalityBuffer
```

```
+559:             ? uint64(block.timestamp) + effectiveFinalityBuffer -
 ↪  finalityBuffer_
560:             : uint64(block.timestamp);
```

Discussion

Oighty

I see the logic here, but would rather be conservative with the finality buffer change so it's ok that it is longer than it needs to be.

Issue L-10: Cross-chain orders remain fillable after being marked `CancelRequested` (cancellation is non-binding) [RESOLVED]

Source: https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/issues/33

Summary

After an order is marked `CancelRequested` on the origin chain, the protocol does not synchronize a "canceled" state to the destination chain, and the origin chain still accepts fill reports while in `CancelRequested`. As a result, solvers can continue filling on the destination chain and still get `tokenIn` released on the origin chain, meaning a "cancel request" cannot prevent subsequent fills (it only starts the refund finality countdown).

Vulnerability Detail

- **EVM side (`evm/src/OrderBook.sol`)**
  - `_requestCancelOrder` only sets the origin order status to `CancelRequested` and records `cancelRequestedAt`; it does not send any cancellation message to the destination chain (`evm/src/OrderBook.sol:305`).
  - On the destination chain, `fillOrder`/`_fillOrder` (when `chainId != orderData_.originChainId`) does not read/validate the origin chain's `localOrders[orderId]` status (and the destination chain does not persist that status either). It only enforces `orderData_.fillDeadline` and `filledAmounts`, so filling remains possible even after the origin order becomes `CancelRequested` (`evm/src/OrderBook.sol:388`).
  - On the origin chain, `reportFill` explicitly allows `order.status == Created || CancelRequested`, so fill reports from the destination chain are still processed and `tokenIn` is released to the solver even after cancellation (`evm/src/OrderBook.sol:478`).
- **SVM side (`svm/programs/order_book`)**
  - `ReportOrderFill.validate` allows fill reporting while `OrderStatus::CancelRequested` (`svm/programs/order_book/src/instructions/report_fill.rs:82`).
  - The destination-chain fill path `FillForeignOrder.validate` only checks filled amounts and has no validation of "origin chain has been canceled" (`svm/programs/order_book/src/instructions/fill.rs:317`).

Impact

- If the intended product semantics are “no further fills after cancel”, the current implementation allows users to be filled after they request cancellation (during the window between cancellation request and refund eligibility), reducing the expected cancelability of orders.
- When a user cancels due to market moves, a solver can still immediately fill on the destination chain, preventing the user from avoiding execution at an outdated price (the user still receives `tokenOut`, but loses the ability to unilaterally stop execution).

Code Snippet

https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/tree/main/liquidity-delivery/evm/src/OrderBook.sol#L478

https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/tree/main/liquidity-delivery/svm/programs/order_book/src/instructions/report_fill.rs#L82

Tool Used

Manual Review

Recommendation

- Clarify protocol semantics: if `CancelRequested` is only meant to “start the refund countdown / signal solvers to stop filling”, document and surface in the UI that “a cancel request does not guarantee preventing cross-chain fills” to avoid user confusion.
- If stronger cancellation guarantees are desired:
  - Have the origin chain reject fill reports after `CancelRequested` (i.e., do not release `tokenIn` while in `CancelRequested`) and introduce a cross-chain cancel message / destination-side cancel flag so destination fills fail once cancellation is observed.
  - Alternatively, introduce a verifiable proof mechanism that can demonstrate “the fill happened before the cancel” (typically complex and costly), to adjudicate the race without fully trusting off-chain components.

Discussion

Oighty

The design intent was ”start the refund countdown / signal solvers to stop filling” as you mentioned. However, as with other issues related to the `finalityBuffer`, this isn't ideal and we're considering alternative designs.

0502lian

This issue has also been fixed in https://github.com/m0-foundation/liquidity-delivery/pull/7

Issue L-11: Incorrect comment in `request_cancel`: `fill_deadline` is not set to the current timestamp [RESOLVED]

Source: https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/issues/34

Vulnerability Detail

- The `handler` comment states: "Set the order status to CancelRequested and the fill deadline to the current timestamp" (`svm/programs/order_book/src/instructions/request_cancel.rs:41`).
- The actual code only updates:
  - `order.status = OrderStatus::CancelRequested` (`svm/programs/order_book/src/instructions/request_cancel.rs:43`)
  - `order.cancel_requested_at = Clock::get()?.unix_timestamp as u64` (`svm/programs/order_book/src/instructions/request_cancel.rs:45`)
- `order.fill_deadline` is never modified; downstream logic (e.g., refund finality checks and cross-chain fill behavior still depending on the original `fill_deadline`) may be misinterpreted due to the incorrect comment.

Impact

Code Snippet

https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/tree/main/liquidity-delivery/svm/programs/order_book/src/instructions/request_cancel.rs#L41

```
// Set the order status to CancelRequested and the fill deadline to the current
↪  timestamp
```

Tool Used

Manual Review

Recommendation

- Fix the comment to match the implementation (e.g., "set CancelRequested and record cancel_requested_at").

Issue L-12: SVM: `fill_foreign_order` can front-run and pre-initialize the `order` PDA, causing the victim's `open_order` to fail (DoS) [RESOLVED]

Source: https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/issues/35

Summary

`open_order` and `fill_foreign_order` share the same `order` PDA seeds (`[ORDER_SEED_PREFIX, order_id]`), but `fill_foreign_order` uses `init_if_needed` for `order` (and does not enforce `origin_chain_id != current_chain_id`). When `dest_chain_id == current chain`, an attacker can call `fill_foreign_order` with the same `order_id` first to create the PDA, causing the victim's `open_order` (which uses `init`) to fail with "account already exists".

Vulnerability Detail

This is essentially “reusing the same PDA address across two different flows (Native/Foreign) + one entrypoint allows `init_if_needed` to create the account”, which enables front-runnable account squatting.

1) On the `open_order` side: the `order` account is created with `init`, and its address is uniquely derived from `ORDER_SEED_PREFIX + compute_order_id(OrderData{...})`.
2) On the `fill_foreign_order` side: the `order` account is created with `init_if_needed`, and its address is derived from `ORDER_SEED_PREFIX + order_id`; additionally, it only constrains `order_data.dest_chain_id == global_account.chain_id` and does not constrain `order_data.origin_chain_id != global_account.chain_id`.
3) If a user sets `dest_chain_id == current chain` in `open_order`, the `order_id` (`compute_order_id(order_data)`) can be computed by anyone who observes the transaction before it lands (since `OrderData` contains sender, nonce, chain IDs, tokens, amounts, etc., which are derivable from the transaction / on-chain state).
4) The attacker front-runs by calling `fill_foreign_order` with the same `order_id` and `order_data`, passes `validate_params` (`computed_order_id == order_id`), and triggers `init_if_needed` to create the `order` PDA.
5) When the victim’s `open_order` executes later, the `order` PDA already exists, so `init` fails and the transaction reverts (“account already exists”), resulting in a DoS against that order; the attacker can repeat this to continuously disrupt the target user.

Impact

An attacker can front-run and squat the `order` PDA to prevent the victim from opening an order, resulting in denial of service (failed transactions, wasted fees, reduced availability).

Code Snippet

https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/tree/main/liquidity-delivery/evm/src/OrderBook.sol#L478

https://github.com/sherlock-audit/2025-12-m-0-limit-order-dec-8th/tree/main/liquidity-delivery/svm/programs/order_book/src/instructions/report_fill.rs#L82

Recommendation

Avoid reusing the same `order` PDA for Native and Foreign flows: use different seeds for `FillForeignOrder` (e.g., include `order_data.origin_chain_id` / an `OrderType` prefix, or a dedicated `FOREIGN_ORDER_SEED_PREFIX`), and/or explicitly enforce `order_data.origin_chain_id != global_account.chain_id` in `fill_foreign_order` (and, depending on intended behavior, consider disallowing `open_order` with `dest_chain_id == current chain`).

Discussion

Oighty

Good find. I think checking `order_data.origin_chain_id != global_account.chain_id` in FillForeignOrder is the simplest fix.

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.